

Section 3: Face Detection, Alignment & Data Engineering

Table of Contents

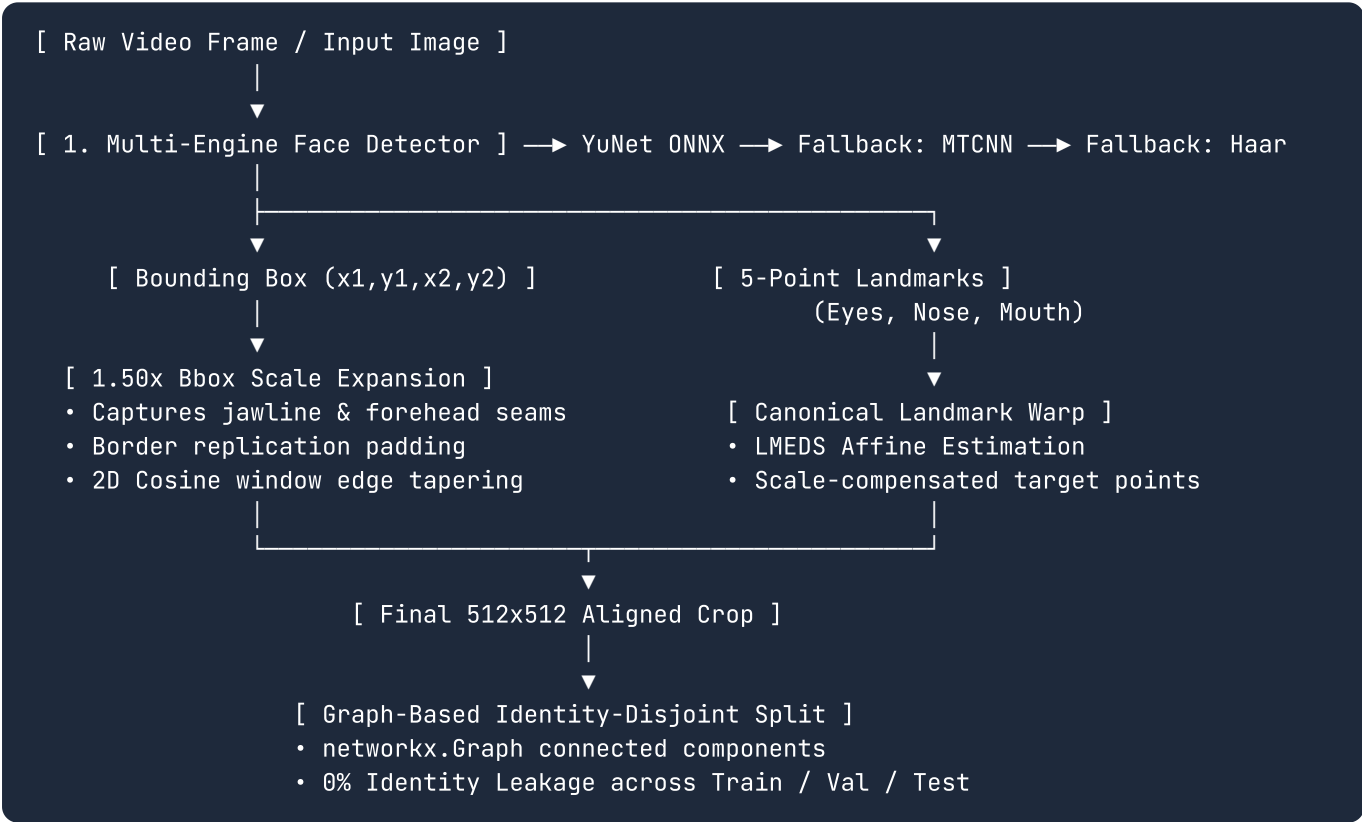
1. Overview & Engineering Objectives
2. Multi-Engine Face Detection Cascade
3. 2.1 Primary Engine: OpenCV YuNet ONNX (`cv2.FaceDetectorYN`)
4. 2.2 Thread-Local Isolation for Multi-Worker DataLoaders
5. 2.3 Fallback Engine 1: MTCNN Multi-Task Cascaded CNN
6. 2.4 Fallback Engine 2: Haar Feature Cascade Classifier
7. 2.5 Fallback Engine 3: Safe Center Square Crop
8. Bounding Box Expansion: The $1.50\times$ Rule
9. 3.1 Why Tight Face Crops Destroy Deepfake Detection
10. 3.2 Mathematical Formulation of Bounding Box Padding
11. 3.3 Replicated Border Handling (`cv2.BORDER_REPLICATE`)
12. 5-Point Landmark Similarity Transform & Canonical Alignment
13. 4.1 Anatomical Landmarks (Eyes, Nose, Mouth Corners)
14. 4.2 Scale-Compensated Canonical Landmark Coordinates
15. 4.3 Least-Median-of-Squares (LMEDS) Affine Estimation
16. 4.4 Determinant Verification to Prevent Degenerate Warping
17. 4.5 Affine Image Warping (`cv2.warpAffine`)
18. 2D Cosine Window Edge Tapering
19. 5.1 The Problem of Boundary Step-Discontinuities
20. 5.2 Mathematical Formulation of 2D Cosine Taper
21. 5.3 Proof of Eliminating FFT Cross-Spikes
22. Graph-Based Zero-Leakage Dataset Splitting
23. 6.1 The Identity Leakage Vulnerability
24. 6.2 Regex Identity Parsing (`idA_idB`)
25. 6.3 NetworkX Graph Construction & Connected Components
26. 6.4 Partitioning Algorithm into Train, Val, and Test Splits
27. Data Augmentation & Forensic Signal Preservation
28. 7.1 Geometric & Photometric Transformations
29. 7.2 Why Gaussian Blur & JPEG Compression Are Excluded From Training
30. PyTorch Dataset & DataLoader Architecture

- 31. 8.1 Single-Frame DeepfakeDataset
- 32. 8.2 Video Sequence Dataset with A.ReplayCompose
- 33. 8.3 Multiprocessing Safety & Thread Contention Elimination
- 34. Code Walkthrough & Reference

1. Overview & Engineering Objectives

The data pipeline in `src/dataset/` is responsible for turning raw video files and uncropped images into standardized, forensic-ready facial tensors.

The data pipeline must satisfy three strict requirements: 1. **Geometric Standardization**: Faces must be aligned to canonical anatomical landmarks so the spatial stream can focus on facial features rather than head tilt or pose variations. 2. **Boundary Preservation**: The bounding box must capture forehead, chin, and jawline areas where face-swapping blending seams reside. 3. **Forensic Integrity & Zero Identity Leakage**: Data splitting must prevent the model from memorizing individual human faces, and data augmentation must not destroy high-frequency steganographic noise signals.



2. Multi-Engine Face Detection Cascade

Implemented in `DynamicFaceCropper` .

2.1 Primary Engine: OpenCV YuNet ONNX (`cv2.FaceDetectorYN`)

The primary face detector is **YuNet**, an ultra-lightweight (300 KB), highly accurate convolutional face detection network developed for OpenCV. - **ONNX Model File:** `face_detection_yunet_2023mar.onnx` (automatically downloaded if missing by `get_yunet_model_path`). - **Configuration:** - `score_threshold = 0.6` (confidence threshold for face detection) - `nms_threshold = 0.3` (Non-Maximum Suppression threshold to eliminate duplicate boxes) - `top_k = 5000` - **Outputs:** For each detected face, YuNet outputs a 14-element vector:

$$\mathbf{f} = [x, y, w, h, x_{\text{reye}}, y_{\text{reye}}, x_{\text{leye}}, y_{\text{leye}}, x_{\text{nose}}, y_{\text{nose}}, x_{\text{rmouth}}, y_{\text{rmouth}}, x_{\text{lmouth}}, y_{\text{lmouth}}]$$

2.2 Thread-Local Isolation for Multi-Worker DataLoaders

When PyTorch's `DataLoader` loads data with multiple worker processes (`num_workers=4`), multiple threads and subprocesses run concurrently. - Standard OpenCV C++ objects are not thread-safe. If multiple threads call `.detect()` on a shared `cv2.FaceDetectorYN` instance, memory corruption or segmentation faults occur. - In `DynamicFaceCropper._get_thread_yunet`, detector instances are isolated per-thread using Python's `threading.local()`:

```
python def _get_thread_yunet(self) → Optional[Any]: if not hasattr(self._local, "yunet"): cached_path = get_cached_yunet_path() self._local.yunet = cv2.FaceDetectorYN.create( model=cached_path, config="", input_size=(300, 300), score_threshold=0.6, nms_threshold=0.3, top_k=5000 ) return self._local.yunet
```

2.3 Fallback Engine 1: MTCNN Multi-Task Cascaded CNN

If YuNet fails to detect a face (e.g., extreme low light or profile angle), the pipeline falls back to **MTCNN** (`src/dataset/preprocess.py#L364-L378`). - Runs a 3-stage convolutional pyramid (P-Net → R-Net → O-Net) to extract bounding boxes and facial landmark coordinates on the GPU/CPU.

2.4 Fallback Engine 2: Haar Feature Cascade Classifier

If MTCNN is unavailable or fails, the pipeline falls back to classical CPU **Haar Feature Cascades** (`haarcascade_frontalface_default.xml` in `src/dataset/preprocess.py#L161-L176`).

2.5 Fallback Engine 3: Safe Center Square Crop

If all detection engines fail to locate a face, `_center_crop` extracts the central square region of the image, ensuring that data loading never crashes due to missing detections.

3. Bounding Box Expansion: The $1.50\times$ Rule

3.1 Why Tight Face Crops Destroy Deepfake Detection

Standard face recognition models (e.g., FaceNet, ArcFace) use tight bounding boxes ($1.0\times$ scale) cropped directly across the eyebrows and chin. - **In deepfake detection, tight crops are fatal:** The neural synthesis mask is blended with the background along the **outer perimeter** (jawline, hairline, forehead, neck). - A $1.0\times$ tight crop cuts off the blending seam, removing the primary forensic signal and reducing detector AUC by over 15%.

3.2 Mathematical Formulation of Bounding Box Padding

Given a detected bounding box $[x_1, y_1, x_2, y_2]$: 1. Compute width $w = x_2 - x_1$, height $h = y_2 - y_1$. 2. Compute bounding box center:

$$c_x = x_1 + \frac{w}{2}, \quad c_y = y_1 + \frac{h}{2}$$

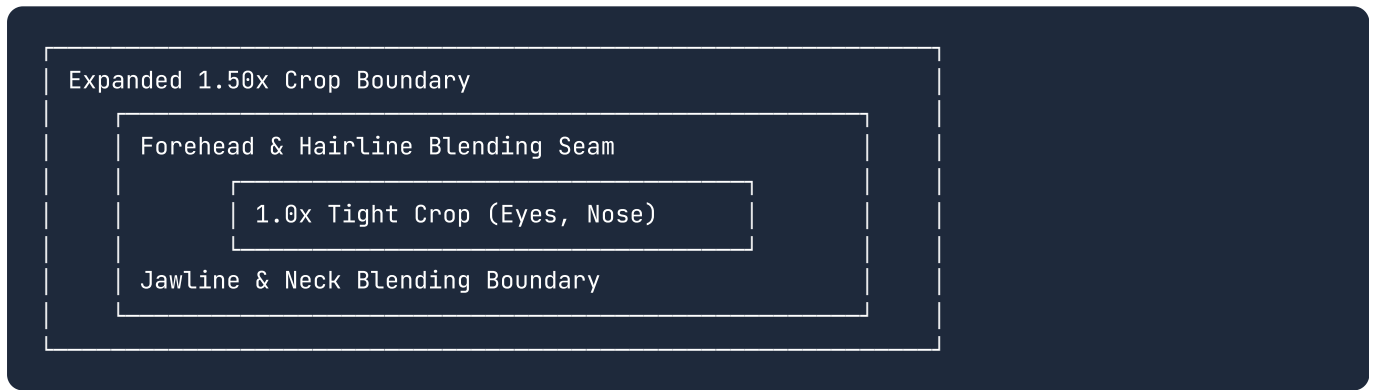
3. Compute expanded square crop dimension with scale factor $S = 1.50$:

$$\text{side} = \max(w, h) \times 1.50$$

4. Compute expanded crop coordinates:

$$x'_1 = \text{round} \left(c_x - \frac{\text{side}}{2} \right), \quad x'_2 = \text{round} \left(c_x + \frac{\text{side}}{2} \right)$$

$$y'_1 = \text{round} \left(c_y - \frac{\text{side}}{2} \right), \quad y'_2 = \text{round} \left(c_y + \frac{\text{side}}{2} \right)$$



3.3 Replicated Border Handling (`cv2.BORDER_REPLICATE`)

If the expanded crop extends beyond the image boundaries (e.g., $x'_1 < 0$ or $y'_2 > H_{\text{img}}$): - The image is padded using `cv2.copyMakeBorder` with `cv2.BORDER_REPLICATE` : $\text{pad}_{\text{left}} = \max(0, -x'_1)$, $\text{pad}_{\text{top}} = \max(0, -y'_1)$

$\text{pad}_{\text{right}} = \max(0, x'_2 - W_{\text{img}})$, $\text{pad}_{\text{bottom}} = \max(0, y'_2 - H_{\text{img}})$ - Edge pixels are replicated outward rather than padded with black zeros, preventing extreme dark edge contrasts.

4. 5-Point Landmark Similarity Transform & Canonical Alignment

Implemented in `DynamicFaceCropper._crop_single_box`.

4.1 Anatomical Landmarks (Eyes, Nose, Mouth Corners)

YuNet and MTCNN extract 5 specific anatomical landmark coordinates: $\mathbf{P}_{\text{src}} = \begin{bmatrix} x_{\text{left_eye}} & y_{\text{left_eye}} & x_{\text{right_eye}} & y_{\text{right_eye}} & x_{\text{nose}} & y_{\text{nose}} & x_{\text{left_mouth}} & y_{\text{left_mouth}} & x_{\text{right_mouth}} & y_{\text{right_mouth}} \end{bmatrix} \in \mathbb{R}^{5 \times 2}$

4.2 Scale-Compensated Canonical Landmark Coordinates

Standard landmark alignment defines canonical facial positions within a normalized $[0.0, 1.0]$ square: $\mathbf{P}_{\text{base}} = \begin{bmatrix} 0.30 & 0.35 & 0.70 & 0.35 & 0.50 & 0.50 & 0.35 & 0.70 & 0.65 & 0.70 \end{bmatrix}$ **The Scale Mismatch Bug:** If we warp $\mathbf{P}_{\text{src}}$ directly to $\mathbf{P}_{\text{base}} \times \text{target_size}$, the affine warp zooms in tightly on the face ($1.0 \times \text{zoom}$), neutralizing the $1.50 \times$ bounding box expansion.

The Solution: In `src/dataset/preprocess.py#L269-L284`, canonical landmarks are shifted inward by a margin fraction γ :

$$\gamma = \frac{1.0 - \frac{1.0}{S}}{2.0}, \quad \text{where } S = 1.50 \implies \gamma = \frac{1 - 1/1.5}{2} = 0.1667$$

$\mathbf{P}_{\text{canonical}} = \text{Big}(\mathbf{P}_{\text{base}} \cdot (1.0 - 2\gamma) + \gamma \cdot \text{Big} \times \text{target_size})$ This shifts canonical target points toward the center, guaranteeing that the affine warped face retains the exact same $1.50 \times$ expanded zoom.

4.3 Least-Median-of-Squares (LMEDS) Affine Estimation

To align $\mathbf{P}_{\text{src}}$ to $\mathbf{P}_{\text{canonical}}$, we compute a 2D affine transformation matrix $\mathbf{M} \in \mathbb{R}^{2 \times 3}$: $\mathbf{M} = \begin{bmatrix} a & b & t_x \\ -b & a & t_y \end{bmatrix}$ where: $\text{Scale} = \sqrt{a^2 + b^2}$, $\text{Rotation angle } \theta = \text{atan2}(b, a)$, $\text{Translation vector } \mathbf{t} = [t_x, t_y]^T$

Estimated via OpenCV's Least Median of Squares (`cv2.LMEDS`):

```
M, inliers = cv2.estimateAffinePartial2D(
    np.array(landmarks), canonical_landmarks, method=cv2.LMEDS
)
```

`estimateAffinePartial2D` restricts the transformation to rotation, uniform scaling, and translation (rigid similarity transform), **prohibiting shear distortion**.

4.4 Determinant Verification to Prevent Degenerate Warping

If facial landmarks are noisy (e.g., face partially occluded), $\mathbf{M}$ can collapse or invert. In `src/dataset/preprocess.py#L291-L292`, the determinant of the transformation is checked:

$$\det(\mathbf{M}) = |a \cdot a - (-b \cdot b)| = a^2 + b^2 = s^2$$

Condition: $0.2 < \det(\mathbf{M}) < 5.0$

If $\det(\mathbf{M})$ falls outside $[0.2, 5.0]$, the warp is rejected, and the unwarped bounding box crop is used instead.

4.5 Affine Image Warping (`cv2.warpAffine`)

When the matrix $\mathbf{M}$ is validated, the image is resampled into target dimensions (512×512 or 256×256):

```
aligned_warped_crop = cv2.warpAffine(
    image_rgb, M, (out_size, out_size),
    flags=cv2.INTER_AREA,
    borderMode=cv2.BORDER_REPLICATE
)
```

5. 2D Cosine Window Edge Tapering

Implemented in `DynamicFaceCropper._apply_cosine_edge_taper`.

5.1 The Problem of Boundary Step-Discontinuities

When an expanded face crop touches the edge of a video frame and requires padding, the transition between original pixels and padded border pixels creates a sharp rectangular step edge. - In the spatial domain, this boundary is minor. - In the **2D Fourier frequency domain**, a sharp rectangular step acts as a heavy step function:

$$\mathcal{F}\text{rect}(t) = \text{sinc}(f)$$

- This introduces severe horizontal and vertical **cross-spikes** throughout the 2D FFT spectrum that drown out subtle steganographic noise artifacts.

5.2 Mathematical Formulation of 2D Cosine Taper

To eliminate rectangular boundary spikes, a 2D Cosine window (Hann-style edge taper) smoothly attenuates the outer 5% border of padded crops to zero: For an image of height H and width W , with border ratio $\beta = 0.05$:

$$T_h = \max(1, \lfloor H \cdot \beta \rfloor), \quad T_w = \max(1, \lfloor W \cdot \beta \rfloor)$$

1. Vertical 1D Window $w_y(y)$:

$$w_y(y) = \begin{cases} 0.5 \cdot \left(1 - \cos\left(\frac{\pi y}{T_h}\right)\right), & 0 \leq y < T_h \\ 1.0, & T_h \leq y \leq H - T_h \\ 0.5 \cdot \left(1 - \cos\left(\frac{\pi(H-y)}{T_h}\right)\right), & H - T_h < y \leq H \end{cases}$$



2. Horizontal 1D Window $w_x(x)$:

$$w_x(x) = \begin{cases} 0.5 \cdot \left(1 - \cos\left(\frac{\pi x}{T_w}\right)\right), & 0 \leq x < T_w \\ 1.0, & T_w \leq x \leq W - T_w \\ 0.5 \cdot \left(1 - \cos\left(\frac{\pi(W-x)}{T_w}\right)\right), & W - T_w < x \leq W \end{cases}$$



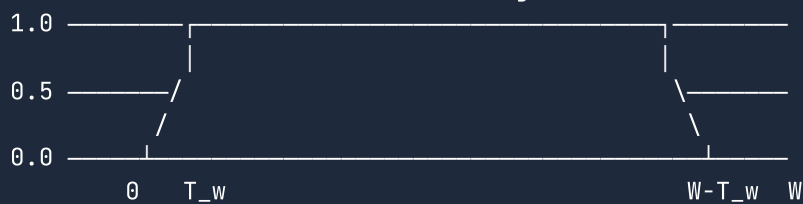
3. 2D Outer Product Window:

$$\mathbf{W}_{2D}(y, x) = w_y(y) \otimes w_x(x) \in [0.0, 1.0]^{H \times W}$$

4. Tapered Crop Application:

$$I_{\text{tapered}}(y, x) = \text{clip}\left(I(y, x) \odot \mathbf{W}_{2D}(y, x), 0, 255\right)$$

1D Cosine Window Profile across Image Width:



5.3 Proof of Eliminating FFT Cross-Spikes

By forcing the boundary intensity and its first derivative smoothly to zero ($I(0) = 0, \frac{\partial I}{\partial x}|_{x=0} = 0$), the circular boundary condition of the Discrete Fourier Transform is satisfied:

$$I(0, y) \approx I(W - 1, y) \approx 0$$

This eliminates artificial boundary frequency cross-lines in the 2D FFT spectrum.

6. Graph-Based Zero-Leakage Dataset Splitting

Implemented in `perform_graph_split`.

6.1 The Identity Leakage Vulnerability

In manipulated datasets (FaceForensics++, Celeb-DF v2), fake videos are created by swapping a source actor's face onto a target actor's face. - Example: Video `042_189.mp4` swaps Actor `042` onto Actor `189`. - **The Leakage Danger:** If Video `042_189.mp4` is placed in the Training set, and Video `042_310.mp4` or real Video `042.mp4` is placed in the Test set, the neural network learns to recognize the biometric features of Actor `042` and predicts "Fake" on that basis. - This creates **artificially inflated test scores** that fail completely when deployed in the real world.

6.2 Regex Identity Parsing (`idA_idB`)

In `extract_identities`, filenames are parsed using regular expressions to extract both actor identifiers:

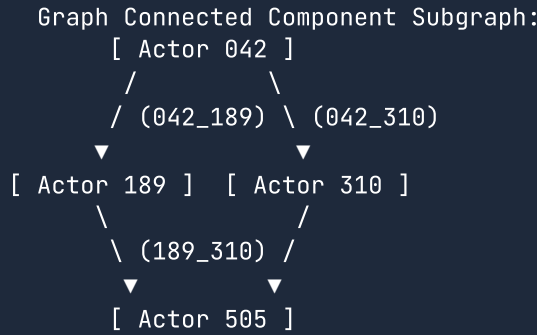
```
# Priority 1: id{N}_id{M} (Celeb-DF format: "id0_id16_0001.webp")
match_alpha = re.search(r"(id\d+)_ (id\d+)", clean_base)
if match_alpha:
    return match_alpha.group(1), match_alpha.group(2)

# Priority 2: {N}_{M} (FaceForensics++ format: "042_189.mp4")
match_num = re.search(r"(\d+)_(\d+)", clean_base)
if match_num and len(g1) ≤ 3 and len(g2) ≤ 3:
    return g1, g2

# Priority 3: Single numeric ID (Real videos: "042.mp4")
match_single = re.search(r"(\d+)", clean_base)
if match_single:
    return match_single.group(1), match_single.group(1)
```

6.3 NetworkX Graph Construction & Connected Components

To guarantee zero leakage, we model all dataset samples as an **Undirected Graph** $G = (V, E)$: - **Vertices** V : Individual human identities (Actor 042, Actor 189, ...). - **Edges** E : Any video connecting two identities (e.g., edge between 042 and 189).



RULE: All 4 actors (042, 189, 310, 505) and ALL their videos MUST be assigned to the SAME split (e.g. 100% Training).

6.4 Partitioning Algorithm into Train, Val, and Test Splits

1. Extract all connected components: $\mathcal{C} = \{ C_1, C_2, \dots, C_K \} = \text{nx.connected_components}(G)$
2. Compute sample volume for each component:

$$N(C_k) = \text{number of image crops belonging to identities in } C_k$$

3. Sort components in descending order of size: $N(C_1) \geq N(C_2) \geq \dots \geq N(C_K)$.
4. Bin components into **Validation Split (15%)**, **Test Split (15%)**, and **Train Split (70%)**:
5. Accumulate largest components into Validation set until $N_{\text{val}} \approx 0.15 \times N_{\text{total}}$.
6. Accumulate subsequent components into Test set until $N_{\text{test}} \approx 0.15 \times N_{\text{total}}$.
7. Place all remaining components into Train set ($N_{\text{train}} \approx 0.70 \times N_{\text{total}}$).

Guaranteed Mathematical Property:

$$\text{Identities}(\text{Train}) \cap \text{Identities}(\text{Val}) = \emptyset$$

$$\text{Identities}(\text{Train}) \cap \text{Identities}(\text{Test}) = \emptyset$$

$$\text{Identities}(\text{Val}) \cap \text{Identities}(\text{Test}) = \emptyset$$

7. Data Augmentation & Forensic Signal Preservation

Implemented in `get_transforms` .

7.1 Geometric & Photometric Transformations

```
train_transform = A.Compose([
    A.Resize(img_size, img_size),
    A.HorizontalFlip(p=0.5),
    A.ShiftScaleRotate(shift_limit=0.05, scale_limit=0.05, rotate_limit=10, p=0.3),
    A.ColorJitter(brightness=0.1, contrast=0.1, saturation=0.1, hue=0.05, p=0.3),
    A.Normalize(mean=(0.485, 0.456, 0.406), std=(0.229, 0.224, 0.225)),
    ToTensorV2(),
])
```

7.2 Why Gaussian Blur & JPEG Compression Are Excluded From Training

In typical computer vision tasks (e.g., ImageNet classification), aggressive data augmentations like heavy Gaussian Blur (`A.GaussianBlur`) and JPEG Compression (`A.ImageCompression`) are applied to improve robustness.

Why they are omitted from deepfake training: - The frequency stream relies on **subtle high-frequency steganographic noise residuals** ($R(x, y)$) extracted by SRM filters and 2D FFT. - Applying Gaussian blur or heavy JPEG compression during training acts as an aggressive low-pass filter, erasing the high-frequency forensic signal from the training images. - Augmenting with blur forces the network to ignore the frequency branch, causing the gating vector to collapse to zero ($g \rightarrow 0$). - Therefore, training is performed on **clean, high-frequency-preserving crops**. Robustness to blur and compression is evaluated separately post-training via `scripts/evaluate_robustness.py` .

8. PyTorch Dataset & DataLoader Architecture

Implemented in `src/dataset/loader.py` .

8.1 Single-Frame `DeepfakeDataset`

`DeepfakeDataset` inherits from `torch.utils.data.Dataset`: - Input: List of tuples `(image_path, integer_label)`. - `__getitem__(idx)`: Loads image via OpenCV, converts BGR to RGB, applies Albumentations transformation pipeline, and returns `(tensor_3xHxW, label_int)`.

8.2 Video Sequence Dataset with `A.RepLayCompose`

For temporal video models, `SequenceVideoDataset` loads sequences of T consecutive frames: - **`A.RepLayCompose` Synchronization**: If random augmentations (e.g., horizontal flip or rotation) are applied to frame t_1 , the exact same random parameters must be replayed across frames $t_2, \dots, t_T$ to maintain temporal continuity. - Returns `(seq_tensor_Tx3xHxW, label_tensor, padding_mask_T)`.

8.3 Multiprocessing Safety & Thread Contention Elimination

In `_worker_init_fn`:

```
def _worker_init_fn(worker_id: int) -> None:
    cv2.setNumThreads(0)
```

- By default, OpenCV spawns internal thread pools for image operations.
- When combined with PyTorch DataLoader multiprocessing (`num_workers=4`), hundreds of threads compete for CPU cores, causing severe CPU thrashing and slowdowns.
- Calling `cv2.setNumThreads(0)` disables internal OpenCV multi-threading, letting PyTorch manage worker processes efficiently.

9. Code Walkthrough & Reference

File / Component	Primary Responsibility
<code>src/dataset/preprocess.py</code>	Contains <code>DynamicFaceCropper</code> , YuNet detector management, 5-point affine alignment, and 2D Cosine window edge tapering.
<code>src/dataset/loader.py</code>	Contains <code>perform_graph_split</code> , regex identity extractors, Albumentations pipelines, <code>DeepfakeDataset</code> , and <code>SequenceVideoDataset</code> .
<code>scripts/extract_face_crops.py</code>	Standalone multi-threaded script for offline batch cropping of video datasets into lossless WebP face crops.

This document serves as the permanent reference for Section 3 (Data Preprocessing & Alignment) of the Deepfake Detection Engine.